



Universidad Técnica Estatal de Quevedo

Facultad de ciencias de la computación

Ingeniería en software

Asignatura:

Ingeniería de requerimientos

Tema:

GA – Exposición Segundo Corte: Herramientas CASE

Estudiante:

Contreras Chavez Kevin German

CL:1251167720

Correo:

kcontrerasc3@uteq.edu.ec

Docente:

PhD. Guerrero Ulloa Gleiston Ciceron

Fecha:

18/07/2026

REGULAR - 2026-2027 PPA

Link GitHub:

<https://github.com/avaldivezoy-hub/PFC-UTeqMarket-MDD-EquipoD>

Índice

1. Resumen	2
2. Fundamentación teórica	2
2.1. MDE, MDA y MDD	2
2.2. Transformaciones y metamodelos	2
2.3. Forward, reverse y round-trip engineering	3
3. Justificación de PowerDesigner	3
4. PFC y subsistema modelado	3
4.1. Contexto, problema y alcance	3
4.2. Modelo UML de origen	4
4.3. Correspondencia con el modelo físico	5
5. Configuración del generador	5
5.1. Decisiones de mapeo	6
6. Proceso de generación	6
7. Verificación del código generado	7
7.1. Compilación y ejecución mínima	7
7.2. Limitaciones identificadas	8
8. Cierre del ciclo: Round Trip	8
9. Repositorio y trabajo colaborativo	9
10. Conclusiones	10
A. Anexo A. Correspondencia detallada entre modelo y código	12
B. Anexo B. Fragmentos representativos	12
C. Anexo C. Estructura mínima del repositorio	13

1. Resumen

UTeqMarket es una plataforma orientada a la compra y venta de productos y servicios entre miembros de la Universidad Técnica Estatal de Quevedo. El problema abordado es la dispersión de publicaciones en redes sociales y grupos de mensajería, donde no existe una validación institucional uniforme ni mecanismos organizados de reputación. Para demostrar un ciclo completo de Model-Driven Development (MDD), se modeló en PowerDesigner el subsistema de publicaciones mediante ocho clases del dominio, asociaciones, cardinalidades, una composición y cuatro enumeraciones. A partir del Modelo Orientado a Objetos se generaron ocho archivos C#; posteriormente se verificaron en Microsoft Visual Studio mediante compilación y una aplicación de consola. También se obtuvo un Modelo Físico de Datos y un script para SQL Server. La primera compilación reveló incompatibilidades de la plantilla, como el uso de `boolean`; tras aplicar ajustes identificados y documentados, el proyecto compiló y la demostración cargó las clases `Usuario` y `Publicacion`. El ciclo se cerró mediante un Round Trip controlado sobre el atributo `idCalificacion`. El resultado evidencia trazabilidad entre requisitos, modelo, código y base de datos, así como las limitaciones reales del generador automático.

2. Fundamentación teórica

2.1 MDE, MDA y MDD

Model-Driven Engineering (MDE) considera los modelos como artefactos de primera clase y no como simples documentos; las implementaciones se derivan de ellos mediante transformaciones automatizadas [1], [2]. Model-Driven Architecture (MDA) organiza el desarrollo en tres niveles: CIM, que expresa contexto y necesidades del negocio; PIM, que describe la solución sin atarla a una plataforma; y PSM, que incorpora decisiones tecnológicas concretas [3]. MDD representa la aplicación práctica de estos principios para producir y evolucionar software a partir de modelos [4].

Ciclo aplicado en UTEqMarket

Requisitos → OOM/UML → C# y PDM → Compilación y ejecución → Round Trip

2.2 Transformaciones y metamodelos

Las transformaciones M2M convierten un modelo en otro, por ejemplo PIM a PSM, mientras que las transformaciones M2T producen artefactos textuales como código, scripts SQL o documentación [5], [6]. UML se define sobre un metamodelo y estandariza clases, atributos, operaciones, asociaciones, multiplicidades y estereotipos [7]. Las plantillas determinan cómo cada elemento del modelo se convierte en una construcción del lenguaje objetivo; su calidad condiciona la fidelidad del código generado [8].

2.3 Forward, reverse y round-trip engineering

Forward Engineering genera código desde el modelo; Reverse Engineering reconstruye un modelo a partir del código; y Round Trip Engineering busca conservar correspondencia entre ambos. La práctica cubrió Forward Engineering y un Round Trip controlado. La literatura reporta beneficios de productividad y trazabilidad, pero también dependencia de herramientas, plantillas y calidad del modelo [9], [10].

3. Justificación de PowerDesigner

PowerDesigner se utilizó como herramienta admisible en la categoría “otras”, porque demostró capacidad real de generar código C# desde un modelo UML y de producir un modelo físico para SQL Server. La selección se vinculó con las necesidades del PFC: lenguaje objetivo C#, uso de Visual Studio, persistencia relacional y necesidad de mantener trazabilidad entre diseño e implementación.

Cuadro 1: Comparación resumida de herramientas evaluadas.

Herramienta	C#	Reverse	BD	Decisión
PowerDesigner	Sí	Sí	Sí	Seleccionada por integrar OOM, PDM, generación C# y SQL Server.
Enterprise Architect	Sí	Sí	Parcial	Alternativa sólida, pero no fue la herramienta instalada para la práctica.
Visual Paradigm	Sí	Sí	Sí	Alternativa con generador robusto; no fue el entorno utilizado por el equipo.

La principal limitación observada fue que la plantilla de C# 2.0 generó esqueletos estructurales y no lógica de negocio completa. Además, exigió ajustes en tipos, enumeraciones y espacios de nombres. Esta limitación se documentó y se distinguió del código producido directamente por la herramienta.

4. PFC y subsistema modelado

4.1 Contexto, problema y alcance

UTeqMarket centraliza publicaciones de productos y servicios para la comunidad UTEQ y utiliza el correo institucional como criterio de pertenencia. El subsistema modelado cubre usuarios, publicaciones, categorías, imágenes, favoritos, contactos, calificaciones y reportes. No incluye interfaz web completa, persistencia implementada en C#, servicios, controladores ni lógica final de los métodos.

Los requisitos funcionales considerados fueron: validar correo institucional, registrar, editar y eliminar publicaciones; buscar y filtrar; visualizar detalle; registrar calificaciones; y marcar publicaciones como vendidas. Los actores son estudiante, docente, personal administrativo y administrador.

4.2 Modelo UML de origen

El OOM contiene ocho clases con el estereotipo «Entity»: Usuario, Publicacion, Categoria, ImagenPublicacion, Favorito, Contacto, Calificacion y Reporte. Se añadieron las enumeraciones RolUsuario, EstadoPublicacion, EstadoReporte y MotivoReporte. El modelo utiliza asociaciones 1..1 a 0..* y una composición entre Publicacion e ImagenPublicacion.

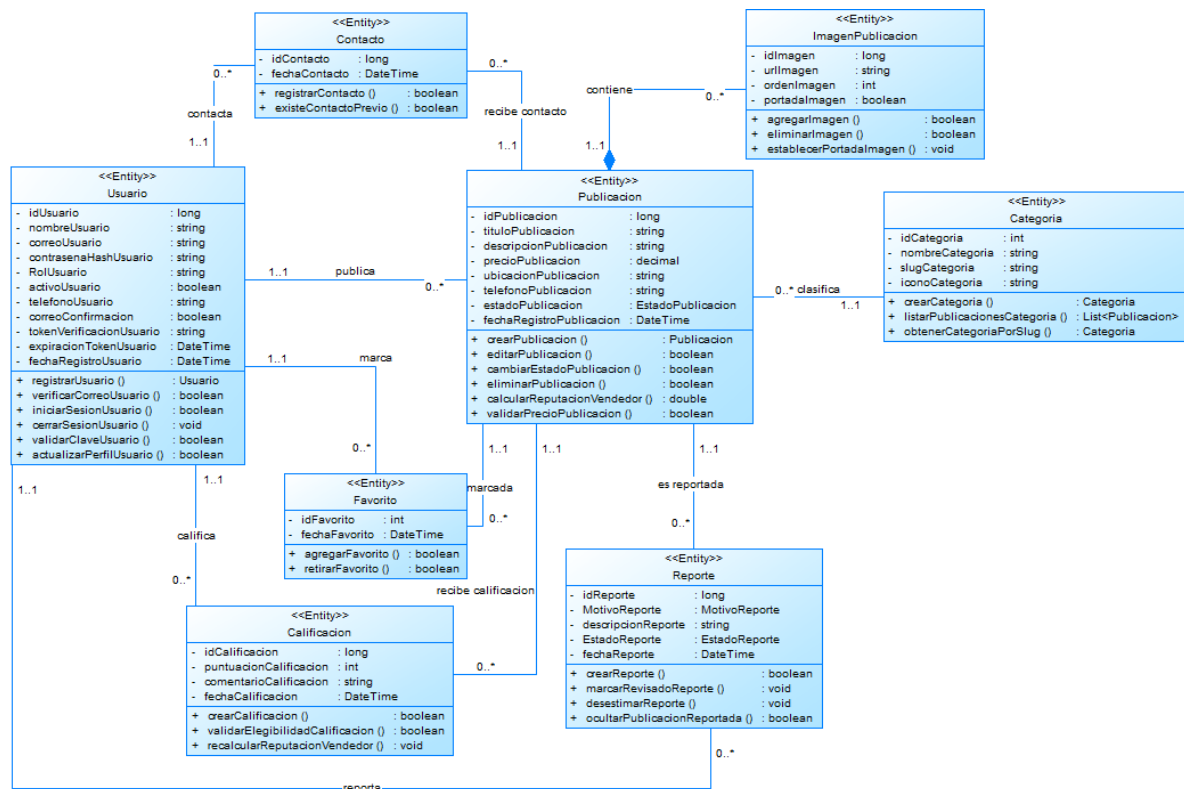


Figura 1: Diagrama de clases UML del subsistema de publicaciones de UTEqMarket.

Cuadro 2: Responsabilidades de las clases principales.

Clase	Responsabilidad
Usuario	Cuenta, correo institucional, rol, activación y perfil.
Publicacion	Producto o servicio, precio, estado y fecha de registro.
Categoria / ImagenPublicacion	Clasificación e imágenes asociadas.
Favorito / Contacto	Interacciones del usuario con una publicación.
Calificacion / Reporte	Reputación e incidencias administrativas.

La validación con *Check Model* no registró errores críticos; las observaciones se revisaron para comprobar nombres, tipos, operaciones, estereotipos y multiplicidades.

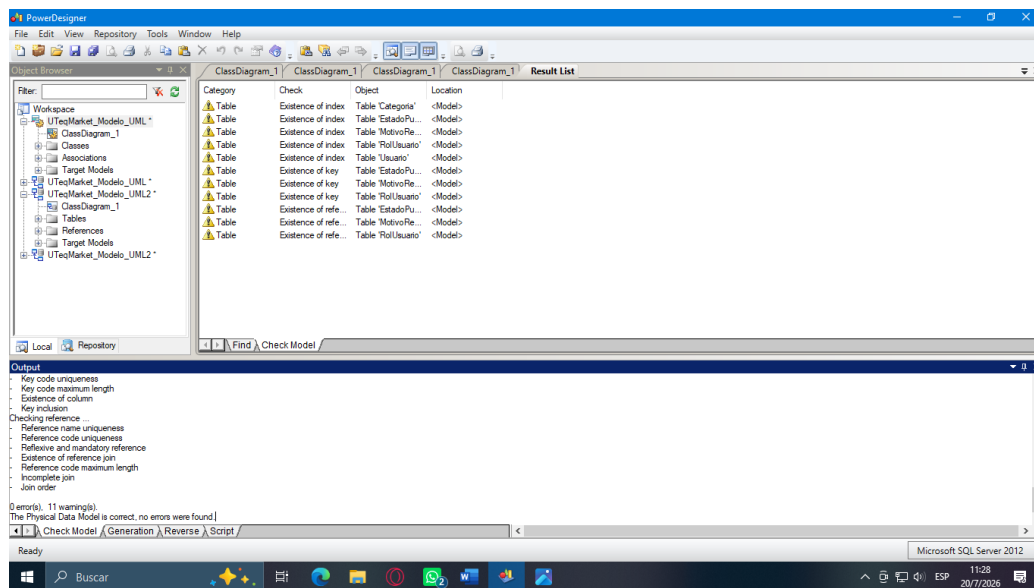


Figura 2: Validación del Modelo Orientado a Objetos en PowerDesigner.

4.3 Correspondencia con el modelo físico

Las asociaciones del OOM se transformaron en referencias y claves foráneas del PDM. Las enumeraciones no se convirtieron en tablas: se implementaron como columnas de texto controladas mediante restricciones.

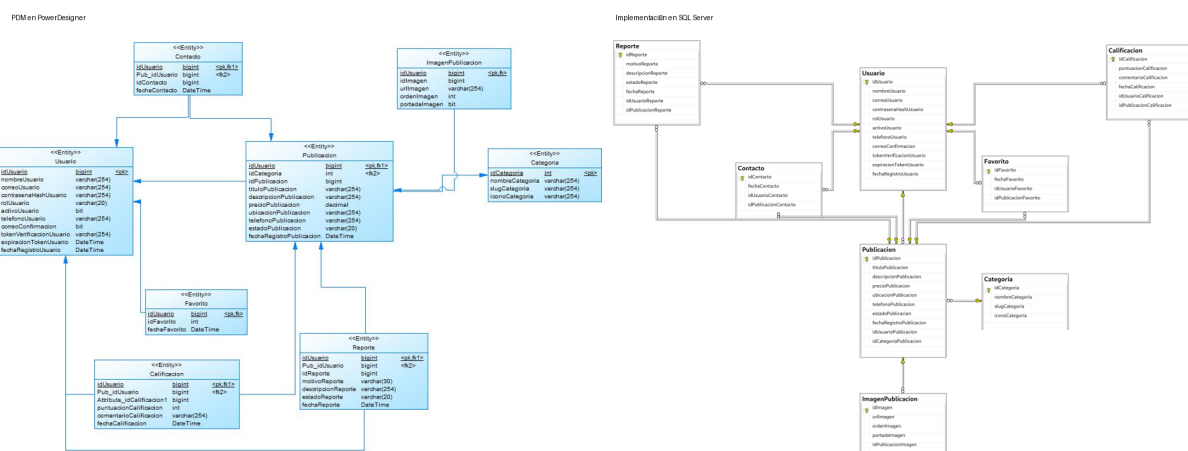


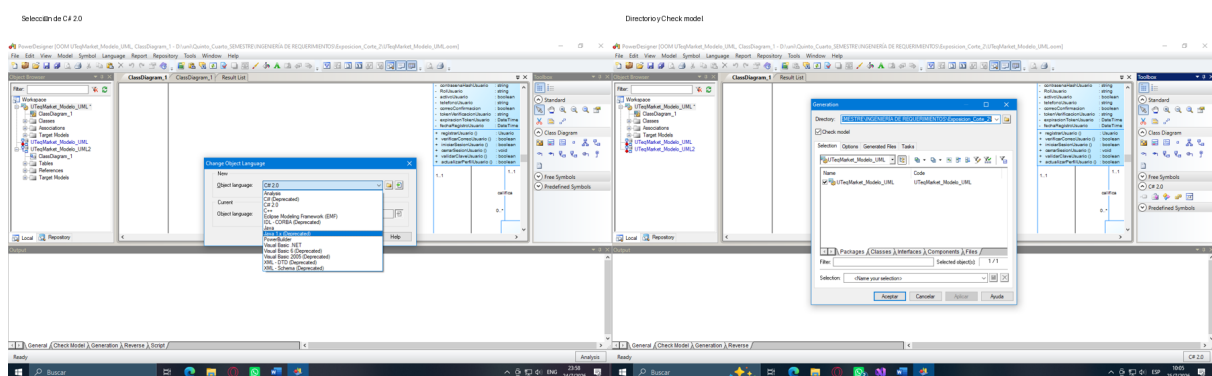
Figura 3: Correspondencia entre el PDM de PowerDesigner y la implementación en SQL Server.

5. Configuración del generador

La generación se ejecutó desde UTEqMarket_Modelo_UML.oom, no desde el PDM. En *Language* → *Change Current Object Language* se seleccionó C# 2.0. Posteriormente se definió el directorio de salida, se mantuvo activa la opción *Check model* y se utilizaron las plantillas predeterminadas.

Cuadro 3: Parámetros documentados del generador.

Parámetro	Configuración aplicada
Modelo de origen	Modelo Orientado a Objetos (.oom).
Lenguaje objetivo	C# 2.0.
Salida	Un archivo .cs por clase, en la carpeta <code>codigo_generado</code> .
Nombres	El nombre de archivo se obtuvo del campo <i>Code</i> de cada clase.
Regeneración	Sobrescritura con copia previa; evidencias separadas en <code>evidencia_round_trip</code> .
Validación	Opción <i>Check model</i> activada.
Plantilla	Definición predeterminada de C# 2.0 de PowerDesigner.

**Figura 4:** Selección del lenguaje y configuración de la generación.

5.1 Decisiones de mapeo

Cuadro 4: Mapeo observado de UML a C#.

Elemento UML	C#	Resultado observado
Clase	public class	Archivo independiente por clase.
Atributo	campo privado	Ejemplo: <code>private long idUsuario;</code>
Operación	método público	Firma generada y cuerpo con <code>NotImplementedException</code> .
Asociación	referencia/colección	No se materializó por falta de roles navegables configurados.
Enumeración	enum	Referenciada, pero no generada en la primera ejecución.

6. Proceso de generación

PowerDesigner procesó las clases y registró la salida en la ventana *Generated Files* y en el panel *Output*. El resultado fueron ocho archivos: `Usuario.cs`, `Publicacion.cs`, `Categoria.cs`, `ImagenPublicacion.cs`, `Favorito.cs`, `Contacto.cs`, `Calificacion.cs` y `Reporte.cs`.

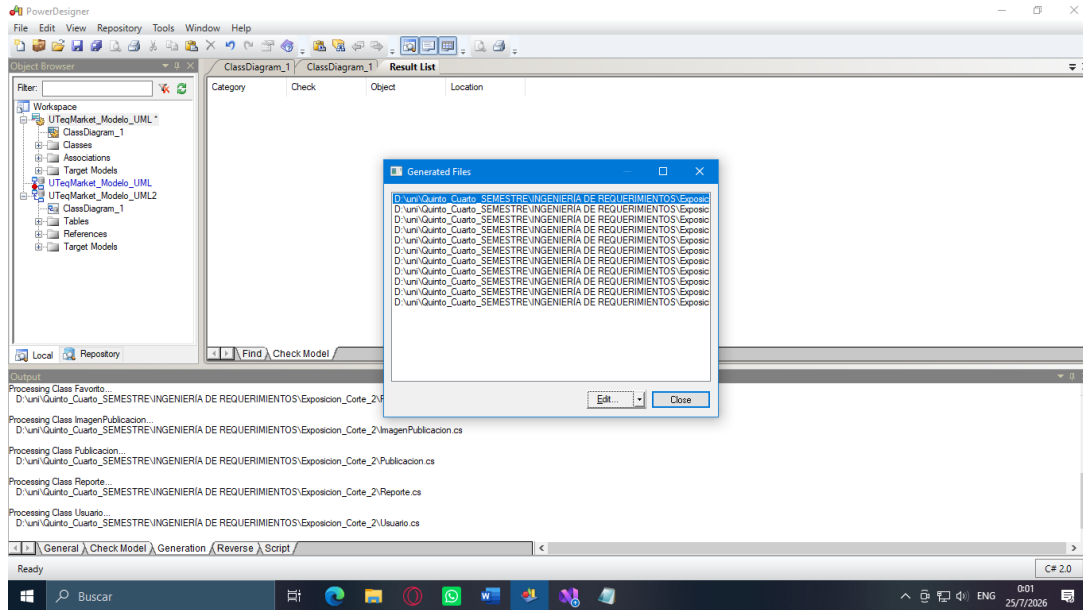


Figura 5: Ventana de archivos generados automáticamente por PowerDesigner.

No se generaron carpetas de controladores, servicios o repositorios porque esas responsabilidades no fueron modeladas. El código producido contiene estructura, campos y firmas, pero no constituye una aplicación funcional completa.

7. Verificación del código generado

7.1 Compilación y ejecución mínima

Los archivos se incorporaron a la biblioteca `UTeqMarket.Modelo`. La primera compilación mostró cuatro errores CS0246 debido al tipo `boolean`, que en C# debe expresarse como `bool`. También se identificaron enumeraciones faltantes y la necesidad de importar `System.Collections.Generic` para `List<Publicacion>`.

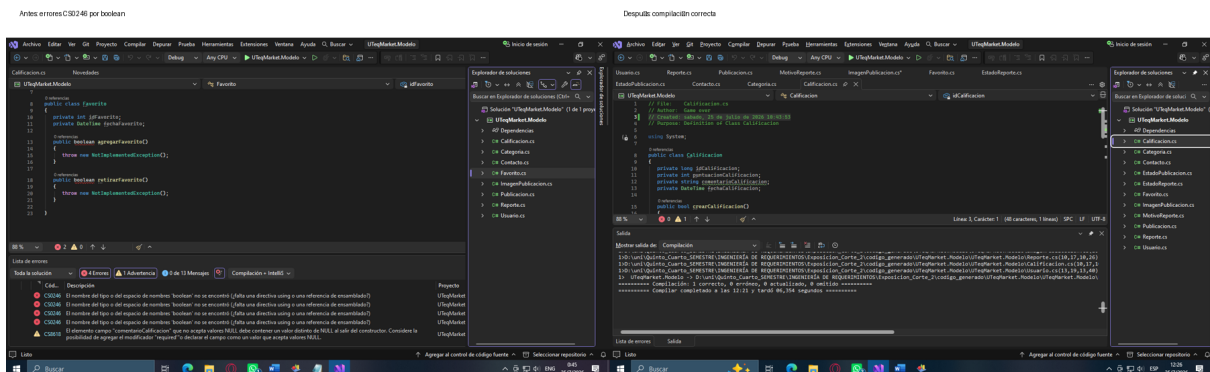


Figura 6: Comparación entre la primera compilación y la compilación final.

Después de aplicar los ajustes documentados, la solución compiló con cero errores. Como la biblioteca no posee punto de entrada, se añadió `UTeqMarket.Demo`; la consola instanció `Usuario` y `Publicacion` y terminó con código 0.

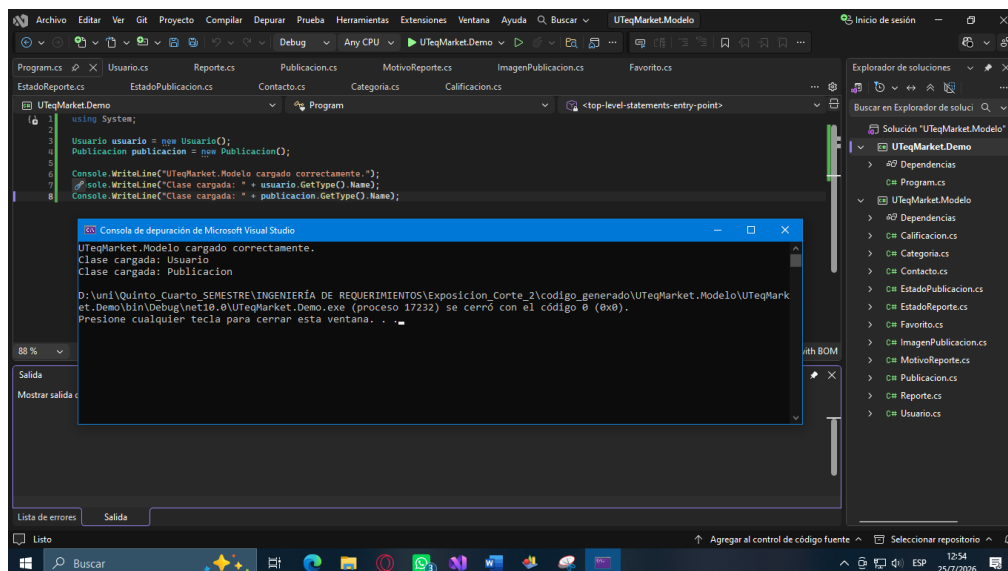


Figura 7: Ejecución de la unidad mínima demostrable en Visual Studio.

7.2 Limitaciones identificadas

Cuadro 5: Limitaciones reales del generador.

Limitación	Tratamiento
boolean no válido	Sustitución por <code>bool</code> en el proyecto compilable.
Métodos sin lógica	Se conservaron como esqueletos; la lógica de negocio queda pendiente.
Enumeraciones faltantes	Se crearon <code>EstadoPublicacion</code> , <code>EstadoReporte</code> y <code>MotivoReporte</code> .
Colecciones	Se añadió <code>using System.Collections.Generic;</code> .
Asociaciones	No se generaron referencias de navegación automáticamente.
Namespace	Los archivos originales no incluyeron un espacio de nombres explícito.

8. Cierre del ciclo: Round Trip

El cambio controlado consistió en corregir el identificador de `Calificacion`. La primera generación produjo `attribute_idCalificacion1`; en el OOM se modificaron *Name* y *Code* a `idCalificacion`, manteniendo tipo `long` y visibilidad privada. Luego se regeneró `Calificacion.cs` en una carpeta separada.

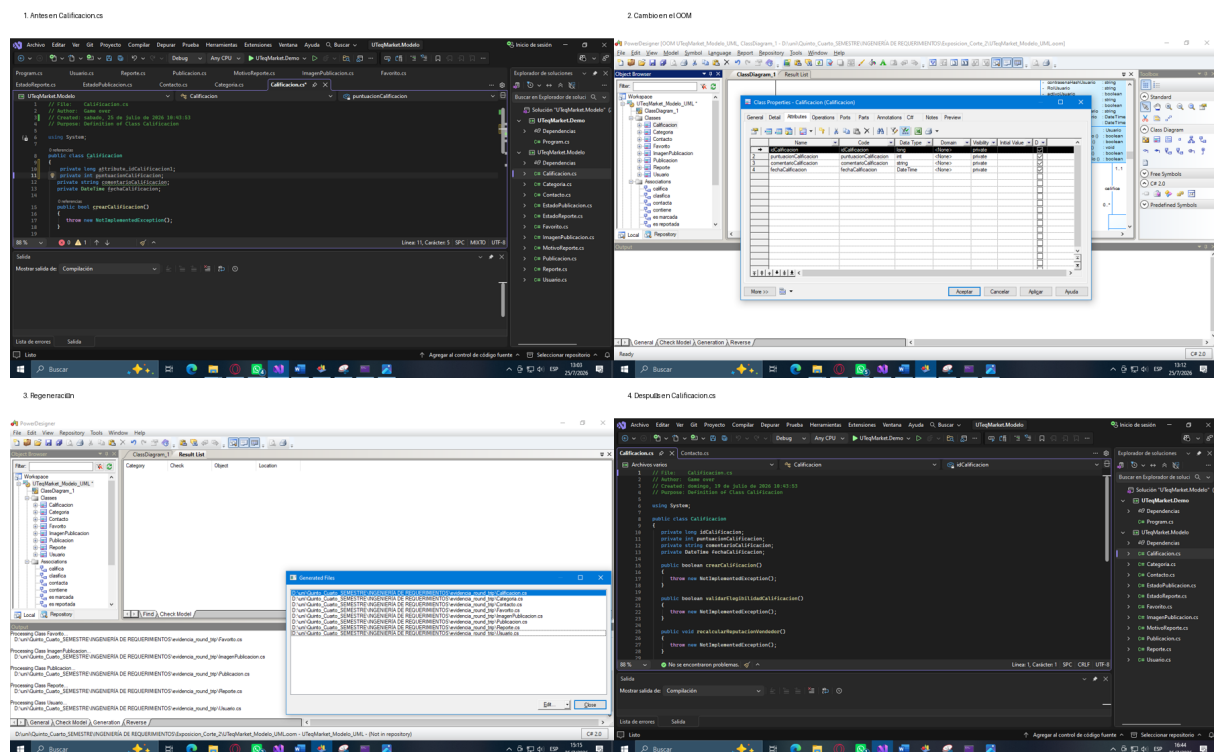


Figura 8: Evidencia antes, cambio en el modelo, regeneración y resultado posterior.

Cuadro 6: Comparación del Round Trip.

Elemento	Antes	Después
Campo	attribute_idCalificacion1	idCalificacion
Tipo	long	long
Visibilidad	private	private
Archivo	Calificacion.cs	Calificacion.cs

La regeneración confirmó que el cambio del modelo se propagó al código. PowerDesigner no evidenció mecanismos de bloques protegidos en esta configuración; por ello, el proceso se realizó sobre una copia separada para evitar sobrescribir correcciones manuales.

9. Repositorio y trabajo colaborativo

El repositorio público organiza documentación, modelos, plantillas o configuración, código generado, evidencias y material de exposición. La evidencia disponible muestra commits diferenciados de kcontreras3-cloud y avaldiviezoy-hub. Este informe documenta únicamente la contribución verificable de Contreras Chávez Kevin German; los nombres y roles completos de los demás integrantes deben coincidir con el README y el historial real antes de la entrega grupal.

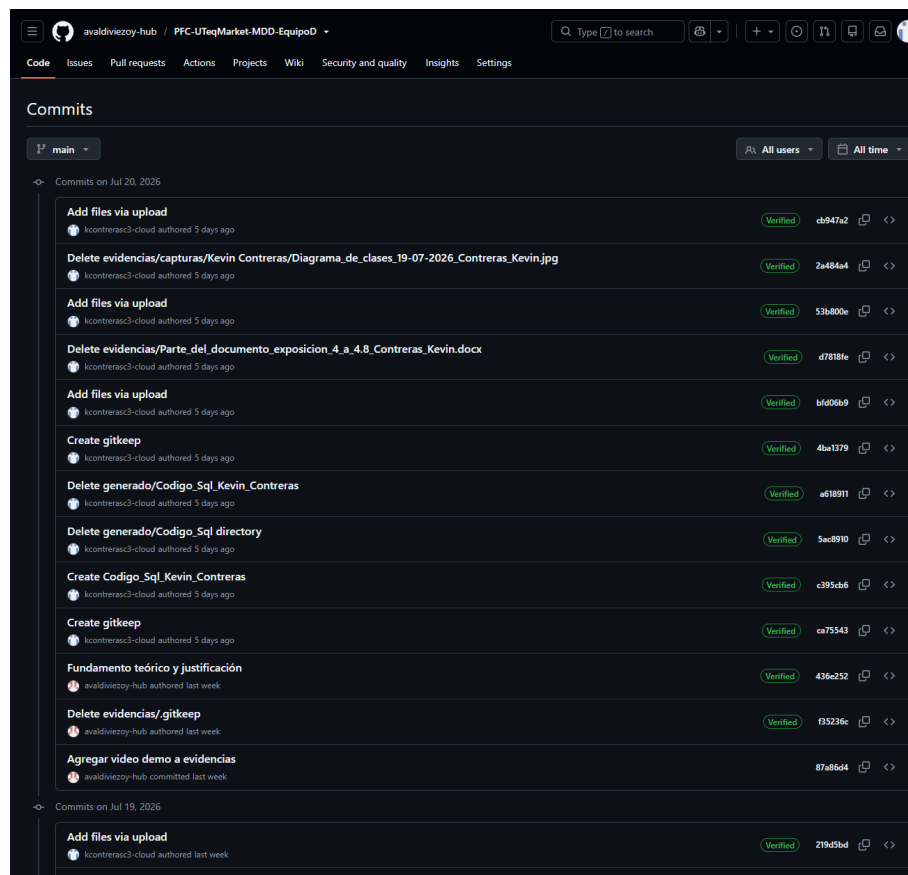


Figura 9: Historial de commits de la rama principal.

Cuadro 7: Commits representativos visibles en GitHub.

Fecha	Autor	Mensaje	Hash
20-jul-2026	kcontrerasc3-cloud	Add files via upload	cb947a2
20-jul-2026	kcontrerasc3-cloud	Delete evidencias/capturas/Kevin Contreras/Diagrama de clases	2a48a44
20-jul-2026	avaldivezoy-hub	Fundamento teórico y justificación	436e252
18-jul-2026	avaldivezoy-hub	Código generado por la herramienta	7a38a92

10. Conclusiones

- MDD permitió usar el modelo UML como fuente estructural para obtener código C# y mantener trazabilidad con el PDM y SQL Server.
- PowerDesigner integró modelado, validación, generación y base de datos, pero la calidad del resultado dependió de la definición correcta de nombres, tipos y roles de asociación.
- La generación automática produjo una base útil, no una solución terminada: la lógica de negocio, capas adicionales y persistencia en C# requieren implementación manual.
- La verificación mediante compilación y consola permitió distinguir claramente

código generado, ajustes necesarios y resultado ejecutable.

- El Round Trip demostró que una corrección en el OOM puede propagarse al archivo generado, siempre que se controle la política de sobrescritura.

Referencias

- [1] D. C. Schmidt, "Guest Editor's Introduction: Model-Driven Engineering," *Computer*, vol. 39, n.º 2, págs. 25-31, 2006. DOI: [10.1109/MC.2006.58](https://doi.org/10.1109/MC.2006.58).
- [2] M. Brambilla, J. Cabot y M. Wimmer, *Model-Driven Software Engineering in Practice*, 2.^a ed. Morgan & Claypool, 2017. DOI: [10.2200/S00751ED2V01Y201701SWE001](https://doi.org/10.2200/S00751ED2V01Y201701SWE001).
- [3] Object Management Group, *Model Driven Architecture Guide, Revision 2.0*, Document ormsc/14-06-01, OMG, 2014.
- [4] B. Selic, "The Pragmatics of Model-Driven Development," *IEEE Software*, vol. 20, n.º 5, págs. 19-25, 2003. DOI: [10.1109/MS.2003.1231146](https://doi.org/10.1109/MS.2003.1231146).
- [5] Object Management Group, *Meta Object Facility 2.0 Query/View/Transformation Specification, Version 1.3*, OMG, 2016.
- [6] Object Management Group, *MOF Model to Text Transformation Language, Version 1.0*, OMG, 2008.
- [7] Object Management Group, *OMG Unified Modeling Language, Version 2.5.1*, Document formal/17-12-05, OMG, 2017.
- [8] E. Syriani, L. Luhunu y H. Sahraoui, "Systematic Mapping Study of Template-Based Code Generation," *Computer Languages, Systems & Structures*, vol. 52, págs. 43-62, 2018. DOI: [10.1016/j.cl.2017.11.003](https://doi.org/10.1016/j.cl.2017.11.003).
- [9] N. Kahani, M. Bagherzadeh, J. R. Cordy, J. Dingel y D. Varro, "Survey and Classification of Model Transformation Tools," *Software and Systems Modeling*, vol. 18, n.º 4, págs. 2361-2397, 2019. DOI: [10.1007/s10270-018-0665-6](https://doi.org/10.1007/s10270-018-0665-6).
- [10] S. Farshidi, S. Jansen y S. Fortuin, "Model-Driven Development Platform Selection: Four Industry Case Studies," *Software and Systems Modeling*, vol. 20, págs. 1525-1551, 2021. DOI: [10.1007/s10270-020-00855-w](https://doi.org/10.1007/s10270-020-00855-w).

A. Anexo A. Correspondencia detallada entre modelo y código

Clase UML	Archivo	Contenido principal
Usuario	Usuario.cs	Cuenta, verificación, inicio y cierre de sesión, actualización de perfil.
Publicacion	Publicacion.cs	Crear, editar, eliminar, cambiar estado, validar precio y reputación.
Categoria	Categoria.cs	Crear categoría, listar publicaciones y buscar por slug.
ImagenPublicacion	ImagenPublicacion.cs	Agregar, eliminar y seleccionar portada.
Favorito	Favorito.cs	Agregar y retirar favoritos.
Contacto	Contacto.cs	Registrar contacto y verificar contacto previo.
Calificacion	Calificacion.cs	Crear calificación, validar elegibilidad y recalcular reputación.
Reporte	Reporte.cs	Crear, revisar, desestimar y ocultar publicación reportada.

B. Anexo B. Fragmentos representativos

Listing 1: Unidad mínima de demostración.

```
using System;

Usuario usuario = new Usuario();
Publicacion publicacion = new Publicacion();

Console.WriteLine("UTeqMarket.Modelo_cargado_correctamente.");
Console.WriteLine("Clase_cargada: " + usuario.GetType().Name);
Console.WriteLine("Clase_cargada: " + publicacion.GetType().Name);
```

Listing 2: Estructura resumida del modelo físico.

```
CREATE TABLE dbo.Calificacion (
    idCalificacion BIGINT NOT NULL,
    puntuacionCalificacion INT NULL,
    comentarioCalificacion VARCHAR(254) NULL,
    fechaCalificacion DATETIME NULL,
    idUsuarioCalificacion BIGINT NOT NULL,
    idPublicacionCalificacion BIGINT NOT NULL,
    CONSTRAINT PK_Calificacion PRIMARY KEY (idCalificacion),
    CONSTRAINT FK_Calificacion_Usuario
        FOREIGN KEY (idUsuarioCalificacion)
        REFERENCES dbo.Usuario(idUsuario),
    CONSTRAINT FK_Calificacion_Publicacion
        FOREIGN KEY (idPublicacionCalificacion)
        REFERENCES dbo.Publicacion(idPublicacion)
);
```

C. Anexo C. Estructura mínima del repositorio

```
PFC-UTeqMarket-MDD-EquipoD/  
|-- README.md  
|-- docs/  
| |-- informe.tex  
| |-- informe.pdf  
| |-- referencias.bib  
| `-- figuras/  
|-- modelo/  
| |-- UTeqMarket_Modelo_UML.oom  
| `-- UTeqMarket_Modelo_Fisico.pdm  
|-- plantillas/  
|-- generado/  
|-- evidencias/capturas/  
`-- exposicion/diapositivas.pdf
```

Datos que deben verificarse antes de entregar

- Versión exacta de PowerDesigner, tomada de *Help* → *About*.
- Nombres completos, roles y commits de todos los integrantes del Equipo D.
- Archivo `informe.tex`, PDF compilado y `referencias.bib` subidos al repositorio.
- README con pasos reproducibles y enlace al video de respaldo.